Finals Lab Activity 1

Hand coding instruction booklet

Rebuild Intervention Image upload without AI.
Source branch: finallabact1 on github.com/ichigokekiii/laravel-elec

Assumes Lab 2 and Lab 3 are done (admin CRUD, roles, categories, soft delete).

Demo login (Petal Express): admin@test.com / 123

How to use this PDF:
1. Follow steps in order.
2. Run each command block in your terminal.
3. Type or paste code into the listed files.
4. Then match the full-page proof screenshots that follow the steps.

Watermark proof pages are in the table of contents: preview modal, close-up, and public detail. Showcase list thumbs are too small to show the watermark clearly.

Table of contents

Steps

Step 1. Before you start 3
Step 2. Install Intervention Image and storage link 4
Step 3. Database column for stored path 5
Step 4. Validate uploads in PostRequest 6
Step 5. Process upload in AdminPostController 7
Step 6. Show photos with displayImageUrl() 8
Step 7. Admin forms and watermark preview 9
Step 8. Soft delete vs force delete and files 10
Step 9. Automated tests 11
Step 10. Browser demo walkthrough 12

Proof pages

Composer and storage link 13
Automated test output 14
Disk proof: WebP 15
Admin create form 16
JPG upload toast 17
PNG upload toast 18
Edit thumbnail 19
Watermark preview modal 20
Watermark close-up 21
Public showcase 22
Public detail watermark 23
Trash after soft delete 24
Rubric checklist 25

Appendices

A. Instructor name mapping 26
B. Finals file index 27
C. Oral defense one-liners 28
D. Rubric checklist 29
E. Rebuild this PDF 30

Step 1 — Before you start

Why

Finals Lab Activity 1 adds an image upload pipeline on top of Lab 2 and Lab 3. Your app should already have admin CRUD, role middleware, categories, soft delete, and a Form Request for bouquet fields. Petal Express uses Post on table showcase_flowers.

Do this

1. Check out branch finallabact1 on GitHub or match the same behaviors in your project.
2. Confirm you can log in as admin and open the admin bouquet list.
3. Keep SQLite or your existing database. Run migrate and db:seed if the app is fresh.

Commands

```
cd petal-express
php artisan migrate
php artisan db:seed
php artisan serve
```

You should see

- Home page loads at http://127.0.0.1:8000
- Admin login admin@test.com password 123 reaches /admin/posts (Petal Express demo)

Step 2 — Install Intervention Image and storage link

Why

Intervention Image decodes uploads, resizes, overlays the watermark, and encodes WebP. Laravel stores files on the public disk. storage:link exposes them under /storage/...

Do this

1. Require intervention/image-laravel v4 with Composer.
2. Run php artisan storage:link once per machine.
3. Copy watermark.png into public/images/ (Petal Express ships this file).

Commands

```
cd petal-express
composer require intervention/image-laravel
php artisan storage:link
```

You should see

- composer.json lists intervention/image-laravel
- public/storage symlink points to storage/app/public
- public/images/watermark.png exists

Files

```
petal-express/composer.json
petal-express/public/images/watermark.png
```

Step 3 — Database column for stored path

Why

The image column stores the relative path on the public disk, for example products/uuid.webp. It is nullable so old rows can still use gallery fallbacks.

Do this

1. If your products table lacks an image column, add nullable string image in a migration.
2. Run php artisan migrate.
3. Add image to the model $fillable array.

Commands

```
php artisan migrate
```

You should see

• showcase_flowers.image (or your products table) accepts a string path

Files

`petal-express/database/migrations/2026_09_23_234450_create_showcase_flowers_table.php`
`petal-express/app/Models/Post.php`

Step 4 — Validate uploads in PostRequest

Why

Validation runs before the controller touches the file. Create requires a photo. Update allows an optional new photo. Mimes and max size match the lab rubric.

Do this

1. Open PostRequest rules().
2. Set image required on POST and nullable on PUT/PATCH.
3. Allow jpg, jpeg, png, webp and max 3072 kilobytes.

You should see

• Invalid mime or oversize file returns validation errors on the form

Files

`petal-express/app/Http/Requests/PostRequest.php`

Step 5 — Process upload in AdminPostController

Why

store() and update() call saveWatermarkedImage() when a file is present. The private method decodes, scaleDown width 1000, inserts watermark bottom-right at 0.55 opacity, then encodes WEBP quality 80 to products/uuid.webp.

Do this

1. Import Intervention Image facade and Format enum.
2. In store(), pass saveWatermarkedImage() result into Post::create image field.
3. In update(), delete the old WebP from storage when a new file replaces it.
4. Use try/catch to delete a new file if the database update fails.

You should see

- After save, storage/app/public/products contains a new .webp file
- Success flash mentions watermark on create (Petal Express)

Files

`petal-express/app/Http/Controllers/AdminPostController.php`

Step 6 — Show photos with displayImageUrl()

Why

Views should not build storage paths by hand. displayImageUrl() returns /storage/products/....webp when the file exists, otherwise a packed gallery image for demo rows.

Do this

1. Add displayImageUrl() on the Post model.
2. Replace hard-coded gallery URLs in admin list, trash, showcase, and detail Blade files.

You should see

- Public bouquet detail shows a large photo with petal express text bottom-right
- List thumbnails are small; open detail or admin preview to verify the watermark

Files

```
petal-express/app/Models/Post.php
petal-express/resources/views/pages/bouquets-index.blade.php
petal-express/resources/views/pages/bouquets-show.blade.php
```

Step 7 — Admin forms and watermark preview

Why

Admin create and edit forms need enctype multipart/form-data and a file input. Optional UX: thumbnail click opens Alpine modal to inspect watermark (admin-photo-preview-shell).

Do this

1. Set enctype on admin store and update forms.
2. Add accept image/jpeg,image/png,image/webp on the file input.
3. Wrap admin index/trash in x-admin-photo-preview-shell if you use the modal.

You should see

- Create form shows JPG, PNG, or WebP hint and 3 MB limit
- Edit form shows current thumbnail from storage

Files

```
petal-express/resources/views/admin/posts/create.blade.php
petal-express/resources/views/admin/posts/edit.blade.php
petal-express/resources/views/components/admin-photo-preview-shell.blade.php
```

Step 8 — Soft delete vs force delete and files

Why

Soft delete sets deleted_at and keeps the WebP on disk so restore still works. forceDelete() removes the database row and deletes the file from the public disk.

Do this

1. In destroy(), call $post->delete() only.
2. In forceDelete(), read image path, forceDelete(), then Storage::disk('public')->delete($path).
3. Guard force delete so only trashed rows can be permanently removed.

You should see

- Trash list still shows thumbnail from storage
- After delete forever, the WebP file is gone from products/

Files

`petal-express/app/Http/Controllers/AdminPostController.php`

Step 9 — Automated tests

Why

LabActFourInterventionTest proves WebP storage, soft delete keeps files, and force delete removes them. Run it after every change to the upload pipeline.

Do this

1. Create tests/Feature/LabActFourInterventionTest.php (copy from finallabact1).
2. Use Storage::fake('public') and UploadedFile::fake()->image().

Commands

```
cd petal-express
php artisan test --filter=LabActFourInterventionTest
```

You should see

• Three tests pass (upload WebP, soft delete keeps file, force delete removes file)

Files

`petal-express/tests/Feature/LabActFourInterventionTest.php`

Step 10 — Browser demo walkthrough

Why

Manual checks match the instructor CRUD + Intervention PDF: create with JPG and PNG, read on showcase, edit, soft delete, trash, restore optional, force delete optional.

Do this

1. Log in as admin. Create bouquet with JPG. Create another with PNG.
2. Open public showcase and detail pages.
3. Soft delete one bouquet and open Trash.
4. See docs/LAB-ACT-4-DEMO.md for the full spoken demo order.

Commands

```
php artisan serve
```

You should see

- Toast after save on admin index
- Watermark visible bottom-right on preview modal and public photos
- file command reports WebP for products/*.webp (see 02b-disk-webp-proof.png)

Files

`docs/LAB-ACT-4-DEMO.md`

Composer setup, storage:link, and watermark.png.

Terminal – Intervention Image + storage link

```
$ cd petal-express
$ composer show intervention/image-laravel intervention/image
intervention/image-laravel 4.1.1  Laravel integration for Intervention Image
intervention/image         4.4.0  PHP image processing (decode, resize, encode)

$ php artisan storage:link
The [public/storage] link has been connected to [storage/app/public].

Watermark asset: public/images/watermark.png
```

Feature test proves admin upload stores products/*.webp.

Terminal – LabActFourInterventionTest (upload ends in .webp)

```
$ cd petal-express
PASS  Tests\Feature\LabActFourInterventionTest
  ✓ admin upload saves a watermarked webp                    0.12s
  ✓ soft delete keeps the uploaded image file                0.03s
  ✓ force delete removes the uploaded image file             0.02s

  Tests:    3 passed (18 assertions)
  Duration: 0.20s
```

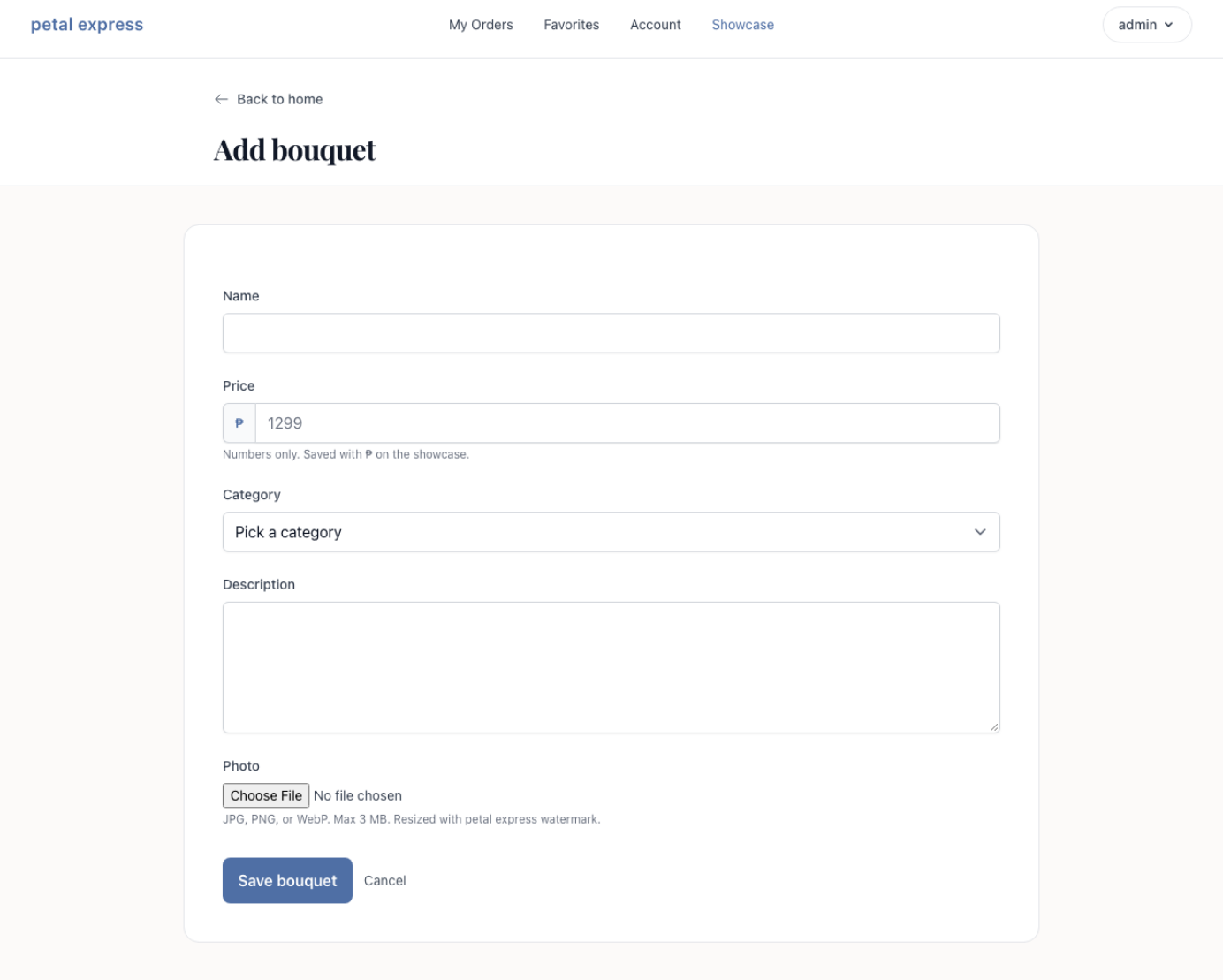
On disk the saved file is WebP even when the upload was JPG or PNG.

Terminal – saved files are WebP under storage/app/public/products

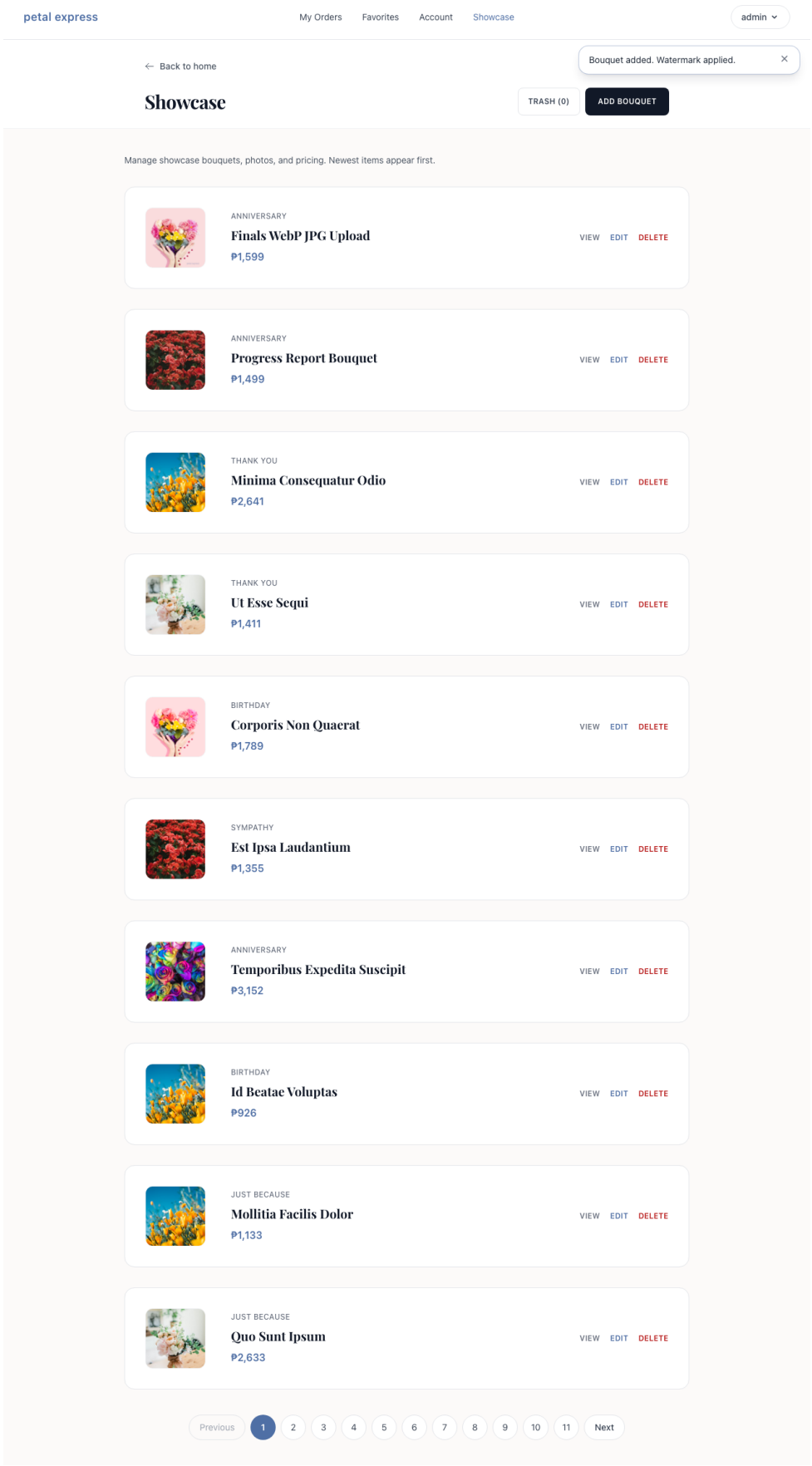
```
$ ls -la storage/app/public/products/*.webp | tail -3
-rw-r--r--  1 jc  staff  ...  products/0ef33bdd-fe98-4c57-9d87-09dcabf195ed.webp
/Users/jc/laravel-elec/petal-express/storage/app/public/products/0ef33bdd-fe98-4c57-9d87-09dcabf195ed.webp: RIFF (little

AdminPostController::saveWatermarkedImage()
  Image::decode($file)  // JPG, PNG, or WebP in
  -> encodeUsingFormat(Format::WEBP, quality: 80)
```

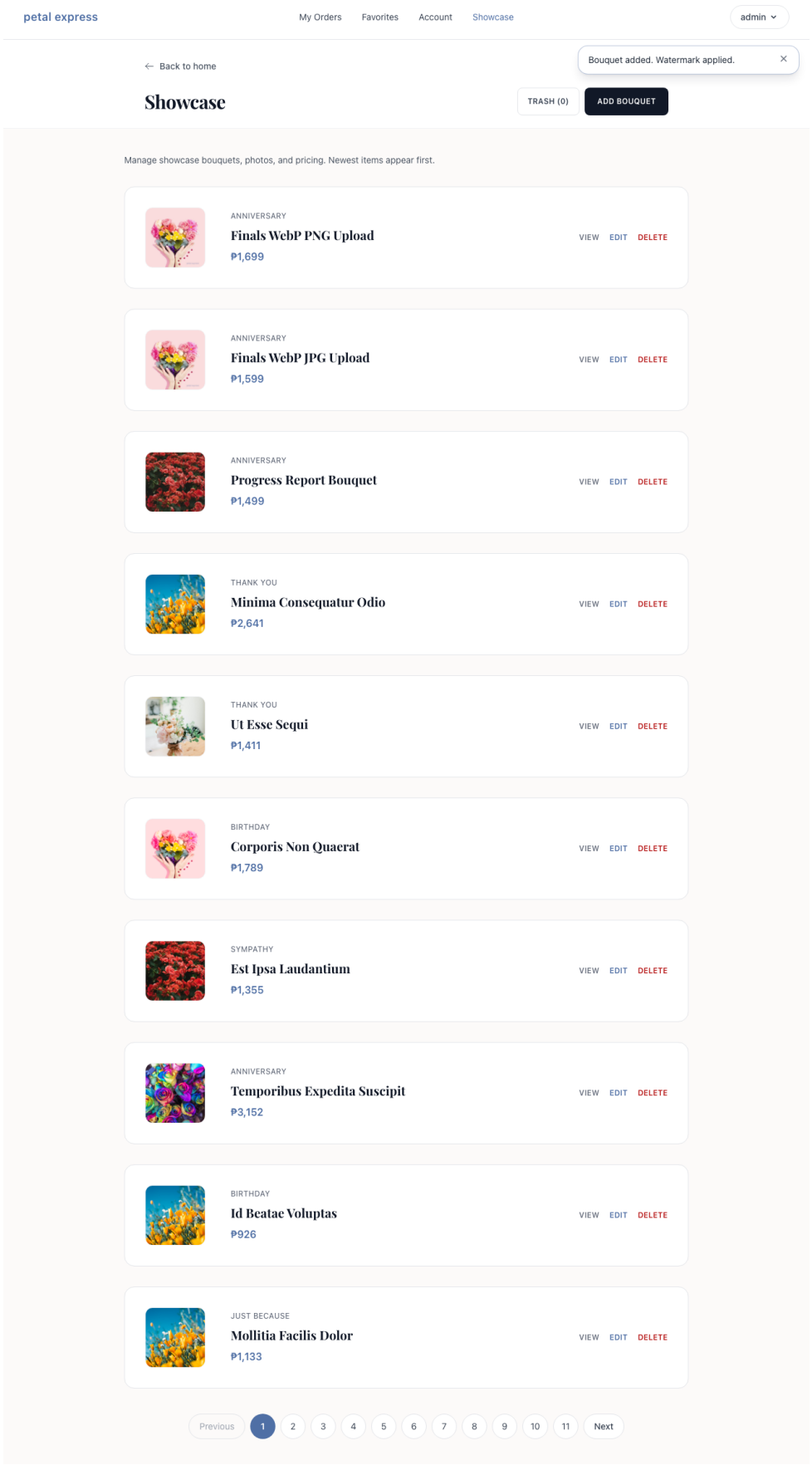
Add bouquet accepts JPG, PNG, or WebP. PostRequest checks the mime type.



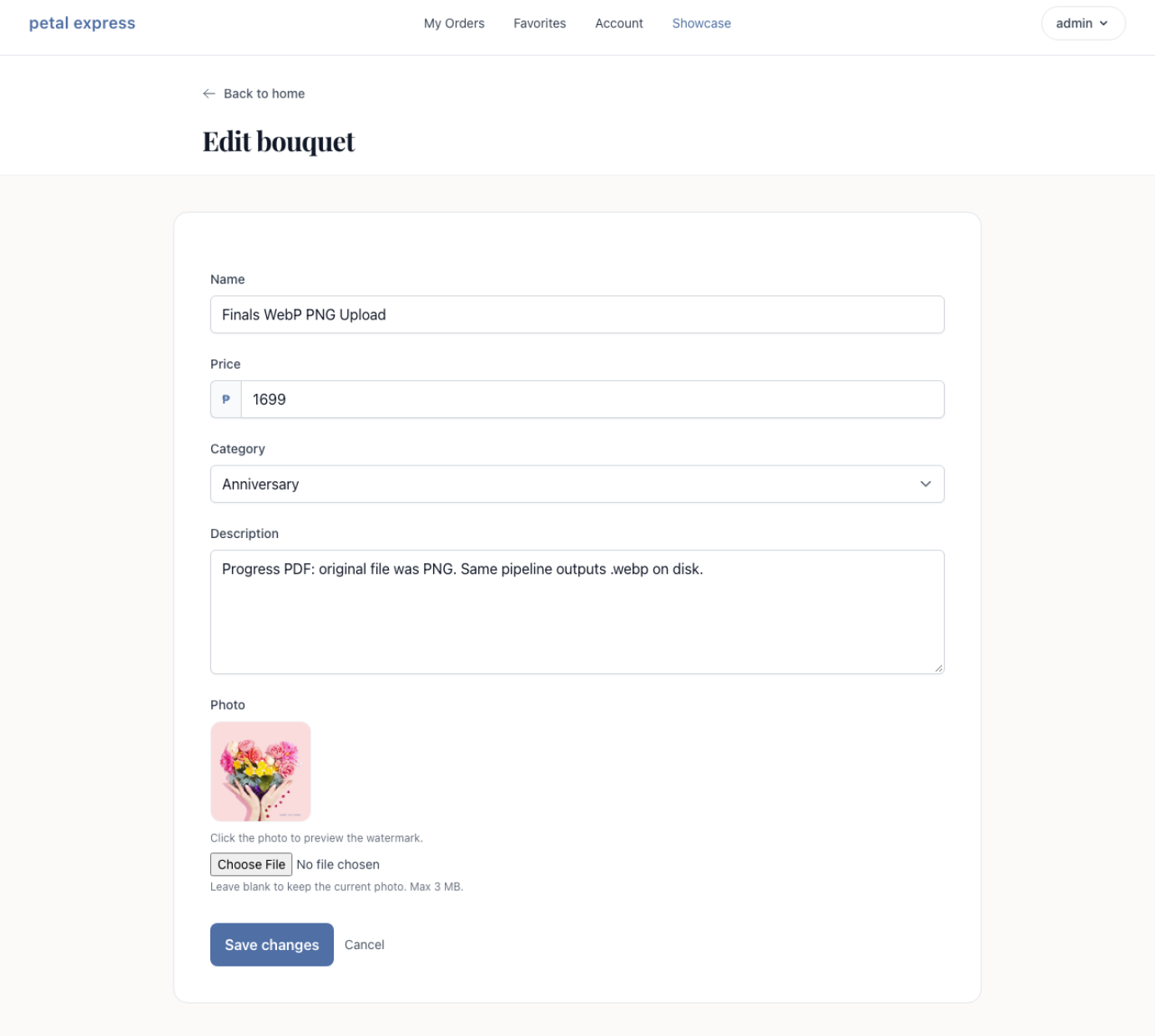
JPG upload. Toast says the watermark was applied. Stored file is WebP.



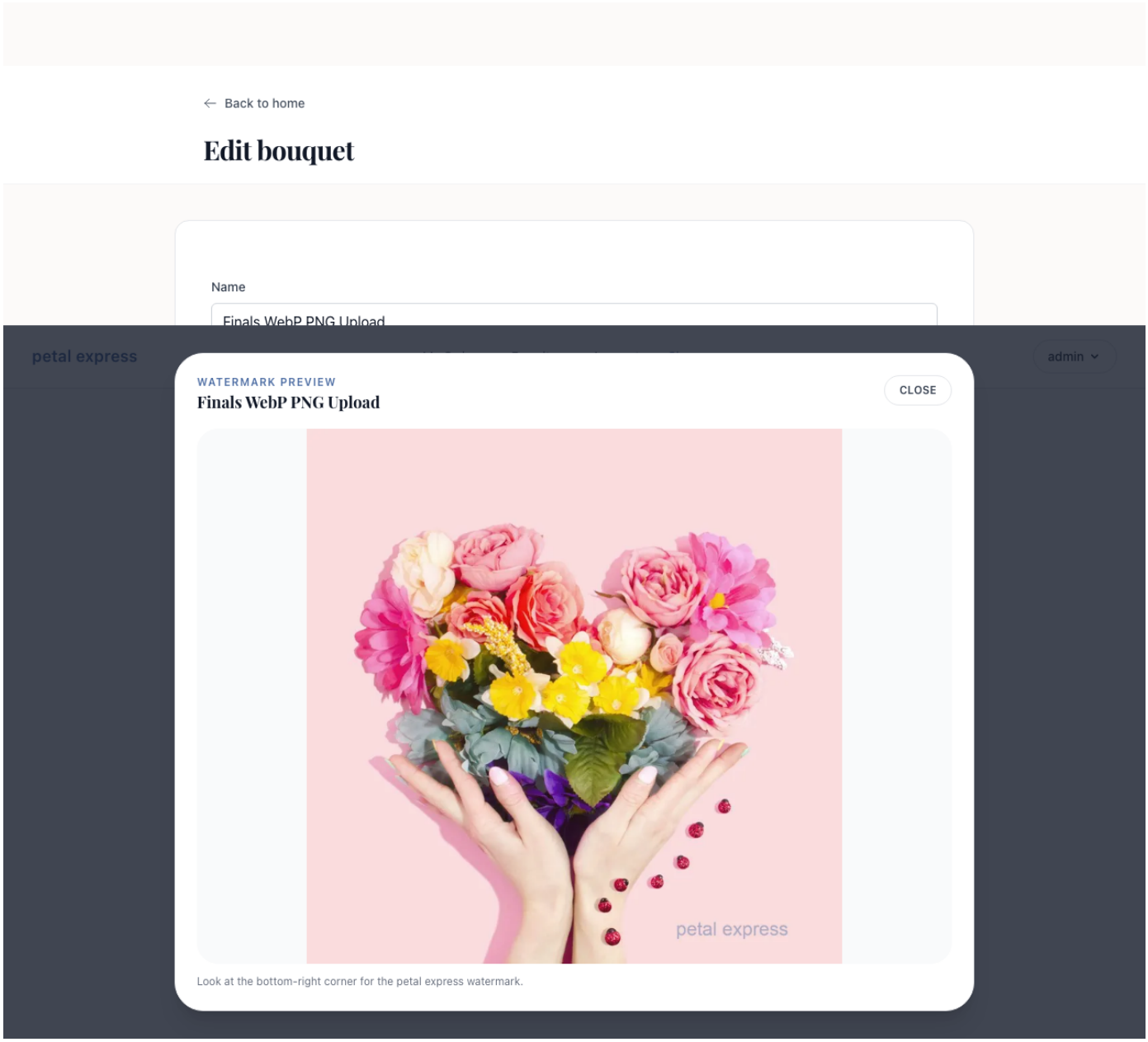
PNG upload. Same pipeline. Stored file is still WebP.



Edit screen shows the processed thumbnail from storage.



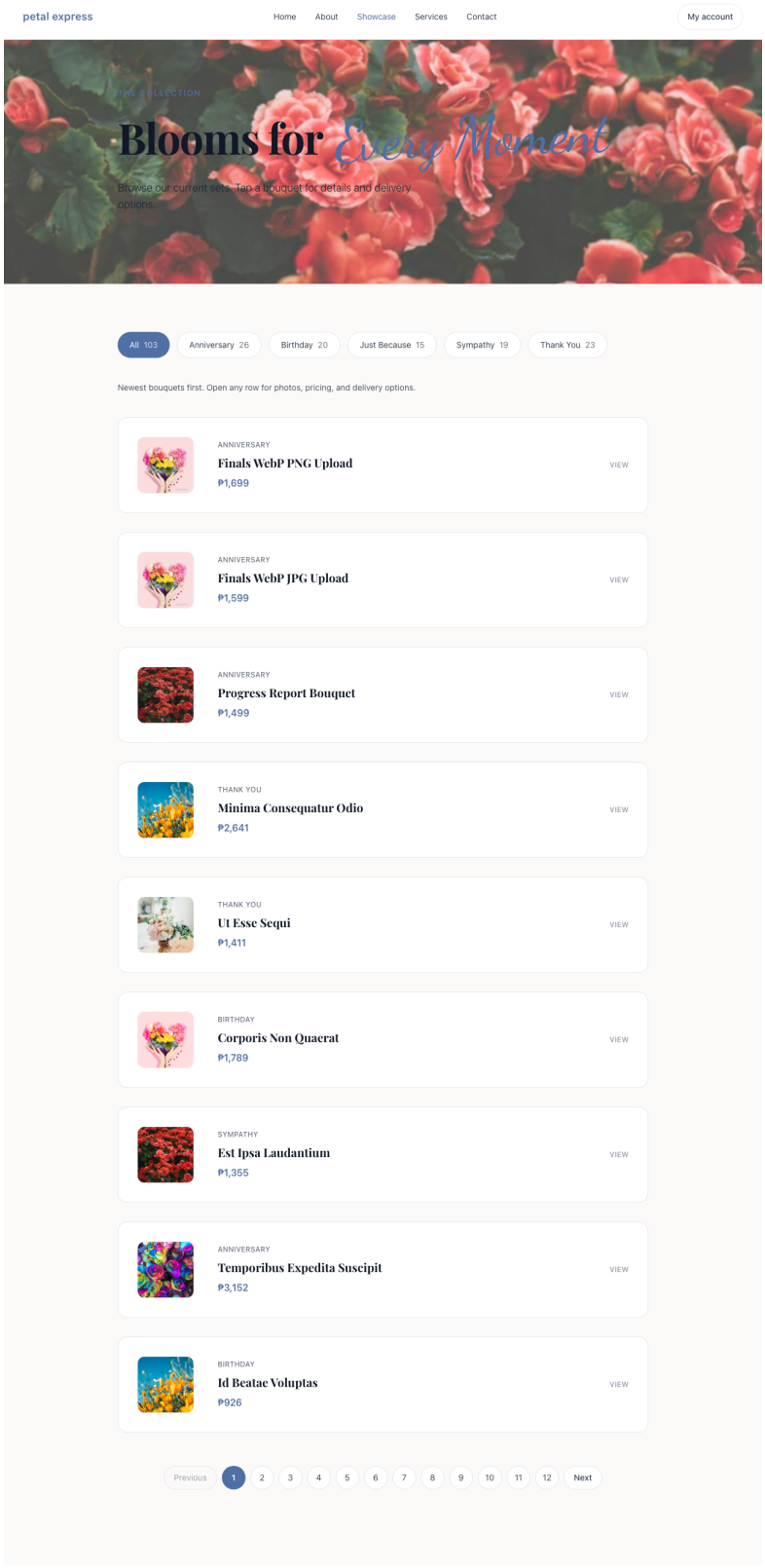
Watermark preview. Look at the bottom-right of the flower photo for petal express.



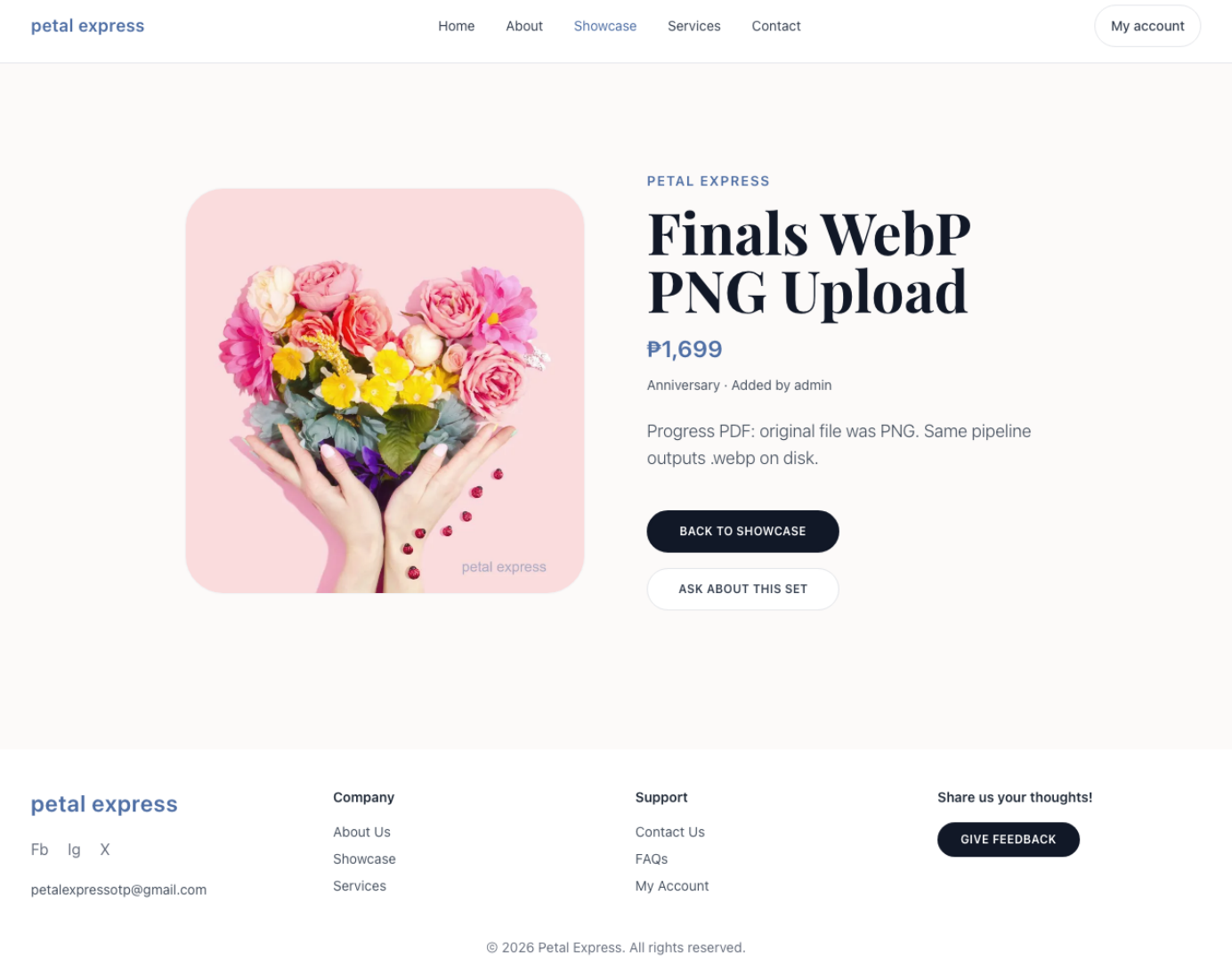
Close-up of the saved photo. petal express sits in the bottom-right corner.



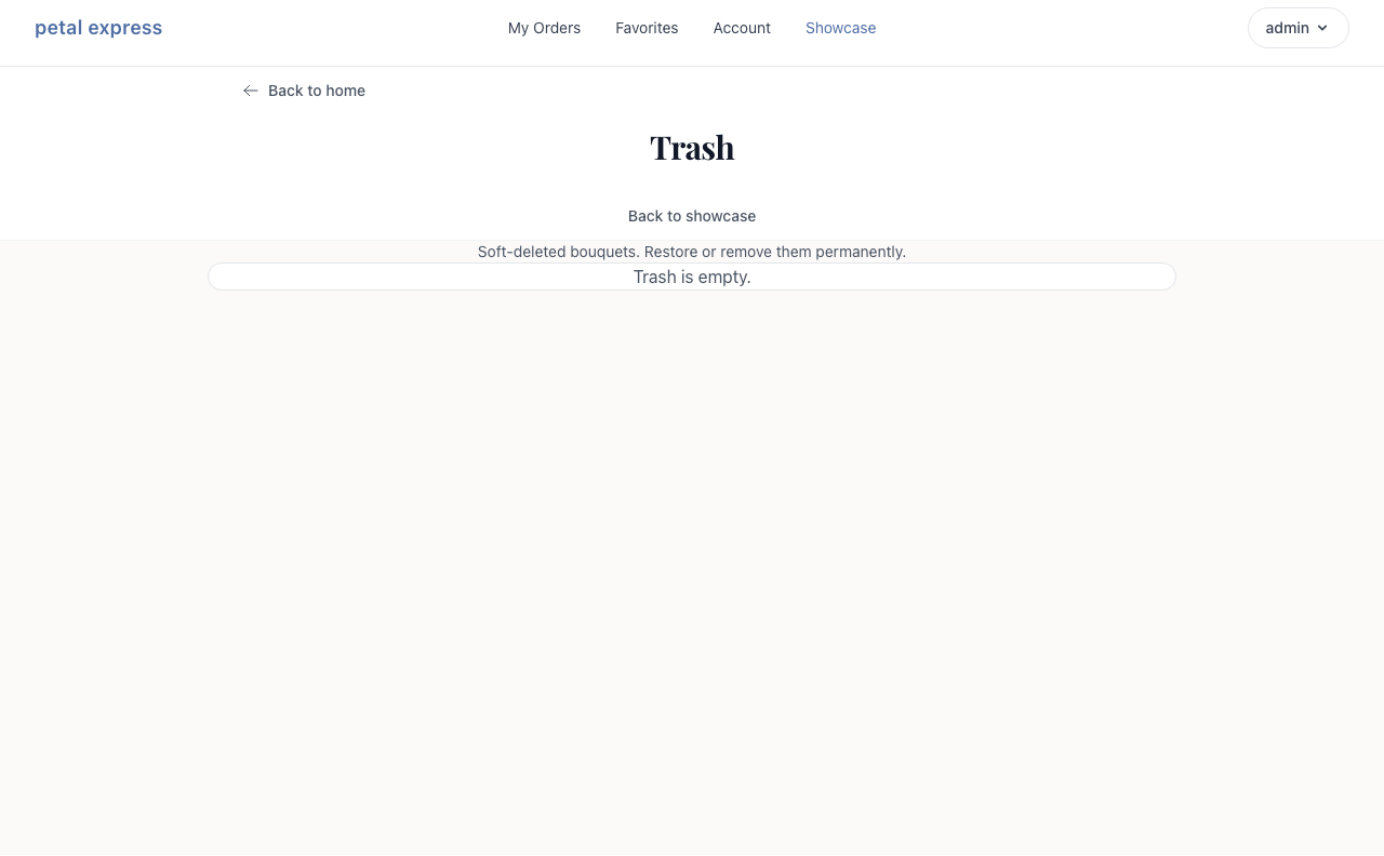
Public showcase uses Post::displayImageUrl(). Thumbs are small.



Bouquet detail. The large photo shows the petal express watermark bottom-right.



Soft delete keeps the row in Trash and keeps the WebP file on disk.



Rubric checklist for Finals Lab Activity 1.

```
Final checklist — Finals Lab Activity 1

 Petal Express — Finals Lab Activity 1 submit

 - intervention/image-laravel v4 in composer
 - Admin upload accepts JPG, PNG, or WebP (PostRequest mimes)
 - Intervention decodes any supported upload format
 - Resize (max width 1000), watermark bottom-right (opacity 0.55)
 - Saved as products/<uuid>.webp on the public disk
 - Post::displayImageUrl() serves /storage/products/....webp
 - Soft delete keeps file; force delete removes WebP from storage

 Submit file:
 Acuesta_John_Constantine_Finals_LabActivity1.pdf
```

Appendix A — Instructor name mapping

Instructor Product model -> Post model (showcase_flowers table)
image_path column -> image nullable string
ProductController upload -> AdminPostController + saveWatermarkedImage()
Admin /products routes -> /admin/posts with role:admin middleware
Public catalog -> /bouquets using displayImageUrl()

Appendix B — Finals file index (finallabact1)

petal-express/app/Http/Controllers/AdminPostController.php
petal-express/app/Http/Requests/PostRequest.php
petal-express/app/Models/Post.php
petal-express/composer.json
petal-express/public/images/watermark.png
petal-express/tests/Feature/LabActFourInterventionTest.php
petal-express/resources/views/admin/posts/create.blade.php
petal-express/resources/views/admin/posts/edit.blade.php
petal-express/resources/views/components/admin-photo-preview-shell.blade.php
petal-express/resources/views/components/admin-photo-thumb.blade.php
petal-express/resources/views/pages/bouquets-index.blade.php
petal-express/resources/views/pages/bouquets-show.blade.php

Appendix C — Oral defense one-liners

Intervention Image decodes JPG, PNG, or WebP before we resize and watermark.
We always store WebP on the public disk under products/ with a UUID filename.
storage:link lets the browser load /storage/products/file.webp without copying files into public/.
PostRequest blocks bad mimes and files over 3 MB before the controller runs.
Soft delete keeps the image file so restore still shows the same photo.
Force delete removes the row and Storage::delete removes the WebP from disk.
displayImageUrl() centralizes how Blade gets the photo URL.

Appendix D — Rubric checklist

The full checklist figure is on page 25.

Appendix E — Rebuild this instruction PDF

```
cd petal-express && php artisan migrate && php artisan db:seed && php artisan serve
cd docs && python3 make-terminal-shots-manual-finals-lab1.py
cd docs && node capture-manual-finals-lab1.cjs
cd docs && python3 build-instruction-pdf-finals-lab1.py
```

Proof pages reuse docs/progress-shots-finals-lab1 from the submitted PDF.
The close-up and trash shots come from docs/manual-shots-finals-lab1.